

EC 4010 – DIGITAL DESIGN

ASSIGNMENT 01

BANDARA H.G.T.D.

2022/E/048

EC 4010

GROUP CG04

2025/02/02

1. Design a 2-to-1 multiplexer using Verilog. The multiplexer should have two data inputs (A, B), one selects input (S), and one output (Y).

- If $S = 0$, $Y = A$
- If $S = 1$, $Y = B$

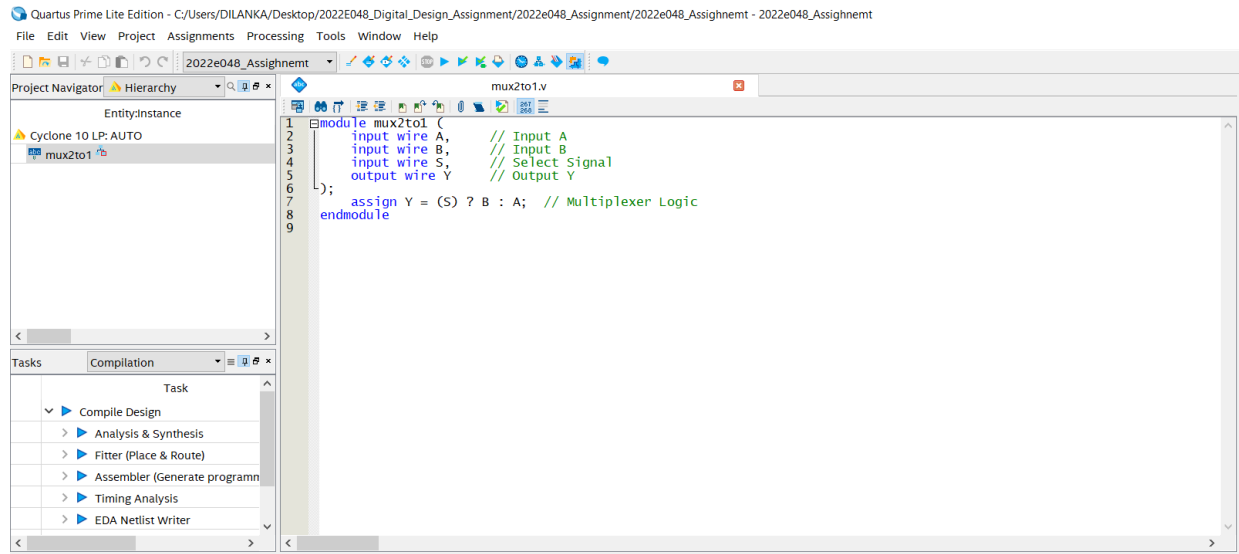


FIGURE 01: VERILOG CODE FOR 2 TO 1 MULTIPLEXER

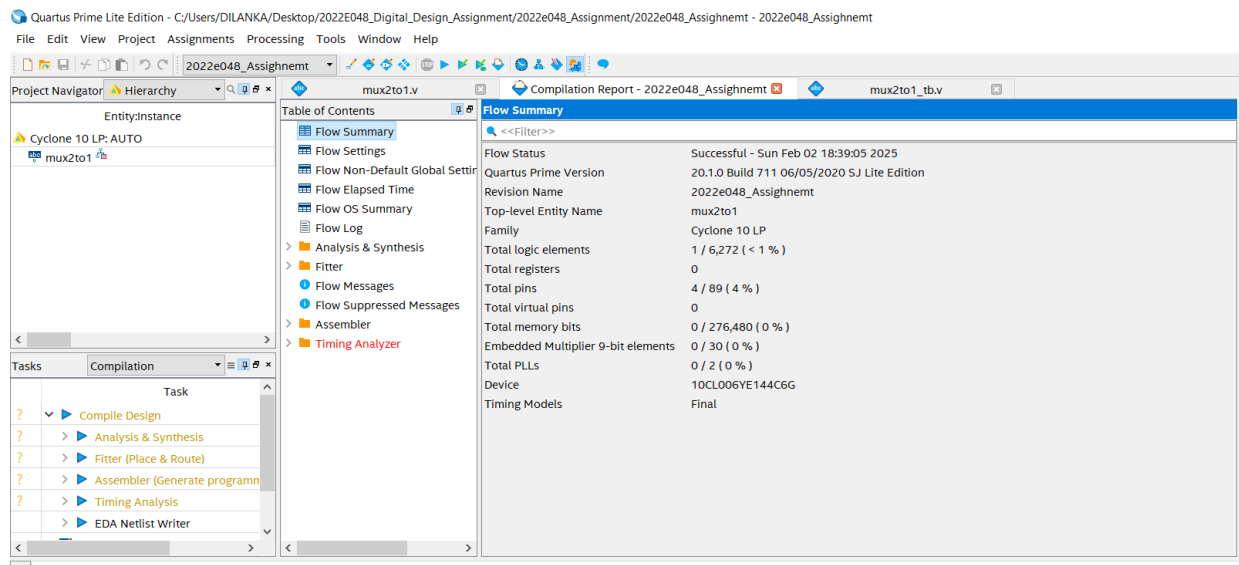


FIGURE 02: ANALYSIS REPORT FOR MULTIPLEXER

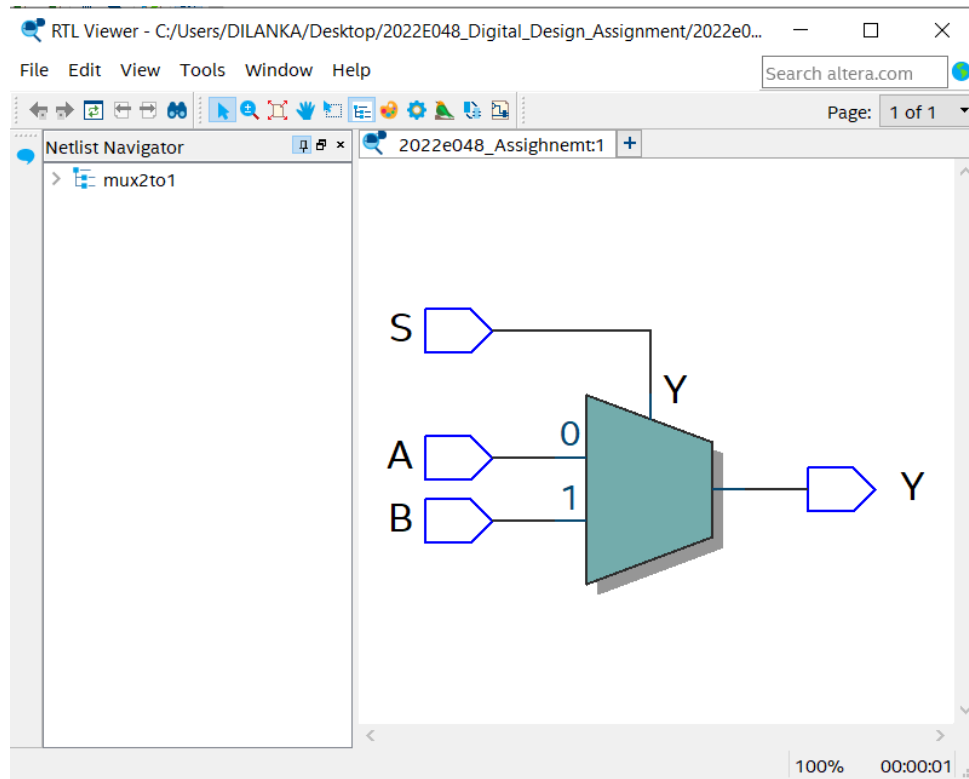


FIGURE 03: NETLIST VIEWER LOGIC CIRCUIT FOR 2 TO 1 MULTIPLEXER

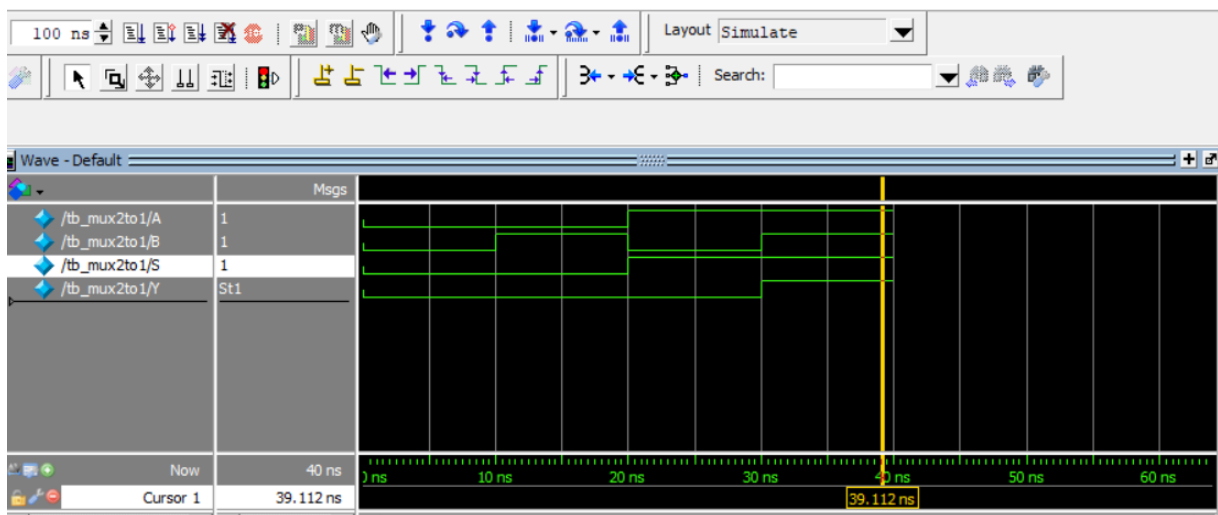
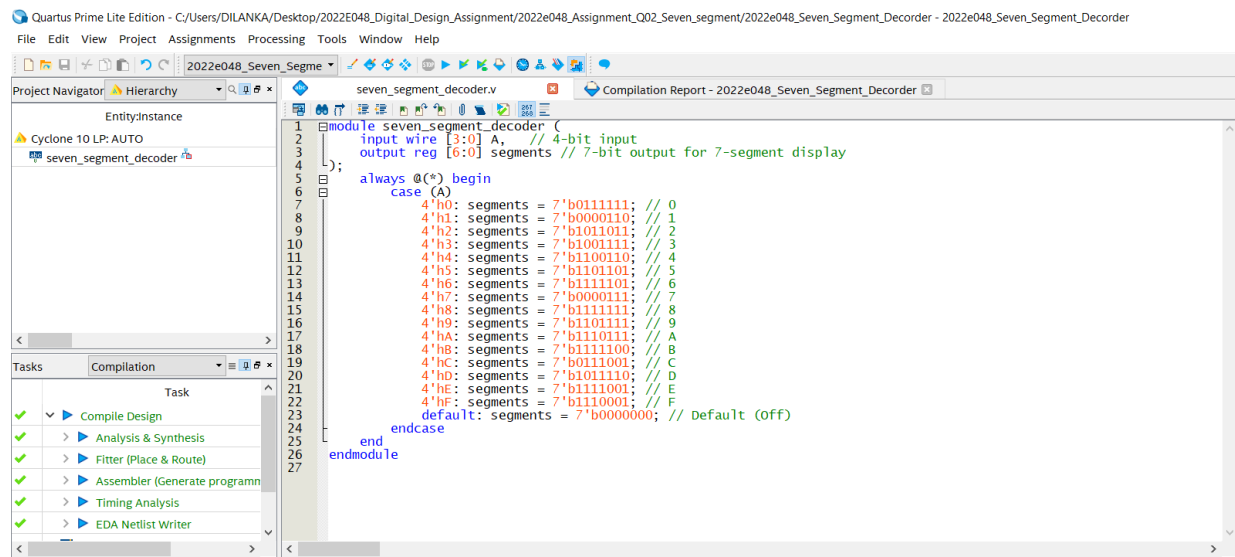


FIGURE 04: OUTPUT FOR 2 TO 1 MULTIPLEXER

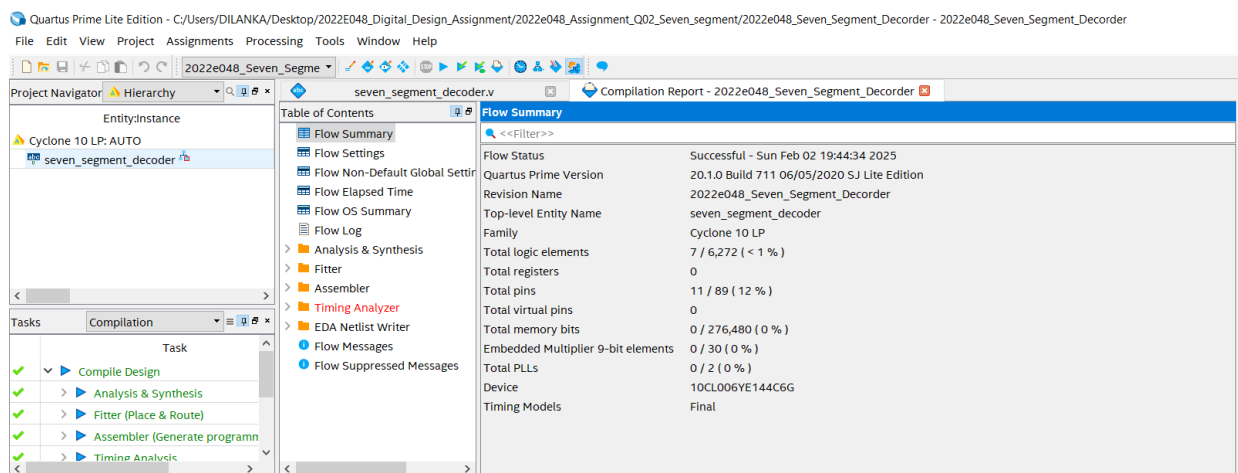
- Design a 7-segment display decoder using Verilog that takes a 4-bit input (A [3:0]) and drives the appropriate segments to display the corresponding hexadecimal digit (0-9, A-F). The output should be in the form of a 7-bit signal (segments [6:0]), where each bit corresponds to a segment on the 7-segment display.



The screenshot shows the Quartus Prime Lite Edition interface. The main window displays the Verilog code for a 7-segment decoder. The code defines a module named `seven_segment_decoder` with a 4-bit input `A` and a 7-bit output `segments`. The code uses a `case` statement to map the 4-bit input to the 7-bit output for hexadecimal digits 0 through F. The output `segments` is a 7-bit signal where each bit corresponds to a segment on the 7-segment display.

```
1 module seven_segment_decoder (
2     input wire [3:0] A, // 4-bit input
3     output reg [6:0] segments // 7-bit output for 7-segment display
4 );
5
6     always @(*) begin
7         case (A)
8             4'h0: segments = 7'b0111111; // 0
9             4'h1: segments = 7'b0000110; // 1
10            4'h2: segments = 7'b1011011; // 2
11            4'h3: segments = 7'b1001111; // 3
12            4'h4: segments = 7'b1100110; // 4
13            4'h5: segments = 7'b1101101; // 5
14            4'h6: segments = 7'b1111101; // 6
15            4'h7: segments = 7'b0000111; // 7
16            4'h8: segments = 7'b1111111; // 8
17            4'h9: segments = 7'b1101111; // 9
18            4'hA: segments = 7'b1110111; // A
19            4'hB: segments = 7'b1111100; // B
20            4'hC: segments = 7'b0111001; // C
21            4'hD: segments = 7'b1011110; // D
22            4'hE: segments = 7'b1111001; // E
23            4'hF: segments = 7'b1111000; // F
24            default: segments = 7'b0000000; // Default (Off)
25        endcase
26    end
27 endmodule
```

FIGURE 05: VERILOG CODE FOR 7 SEGMENT DECODER



The screenshot shows the Quartus Prime Lite Edition interface with the Analysis Report open. The report provides a summary of the compilation process and the resulting design parameters.

Flow Summary	
Flow Status	Successful - Sun Feb 02 19:44:34 2025
Quartus Prime Version	20.1.0 Build 711 06/05/2020 SJ Lite Edition
Revision Name	2022e048_Seven_Segment_Decoder
Top-level Entity Name	seven_segment_decoder
Family	Cyclone 10 LP
Total logic elements	7 / 6,272 (< 1 %)
Total registers	0
Total pins	11 / 89 (12 %)
Total virtual pins	0
Total memory bits	0 / 276,480 (0 %)
Embedded Multiplier 9-bit elements	0 / 30 (0 %)
Total PLLs	0 / 2 (0 %)
Device	10CL006YE144C6G
Timing Models	Final

FIGURE 06: ANALYSIS REPORT 7 SEGMENT DECODER

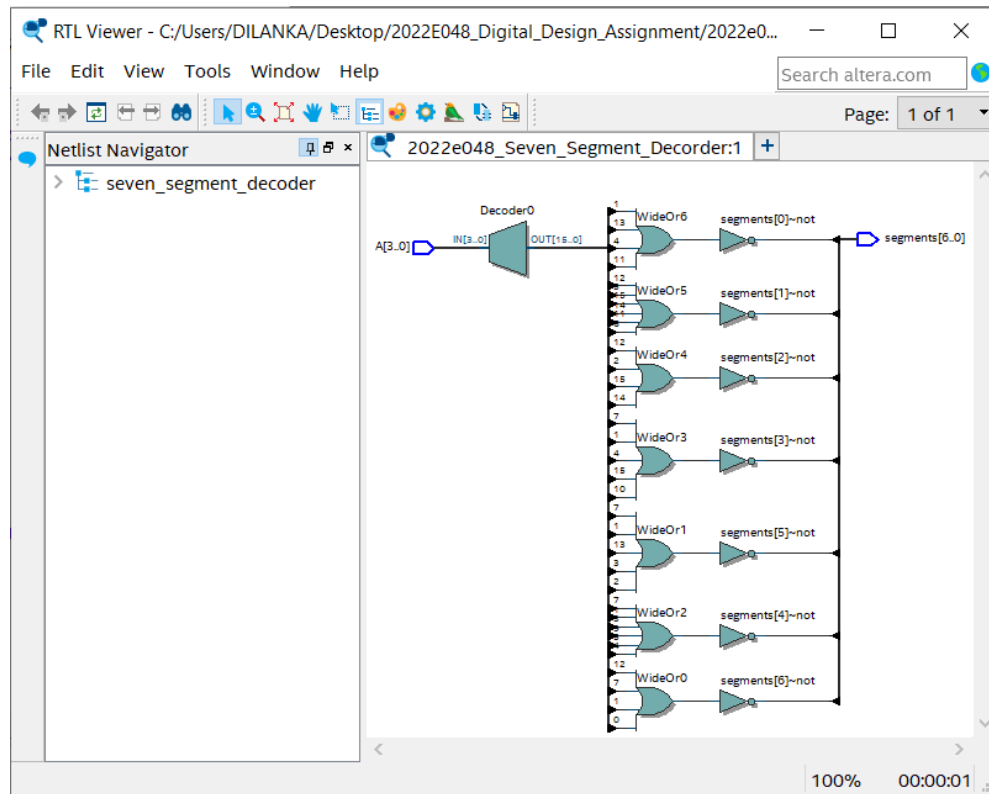


FIGURE 07: LOGIC CIRCUIT FOR 7 SEGMENT DECODER

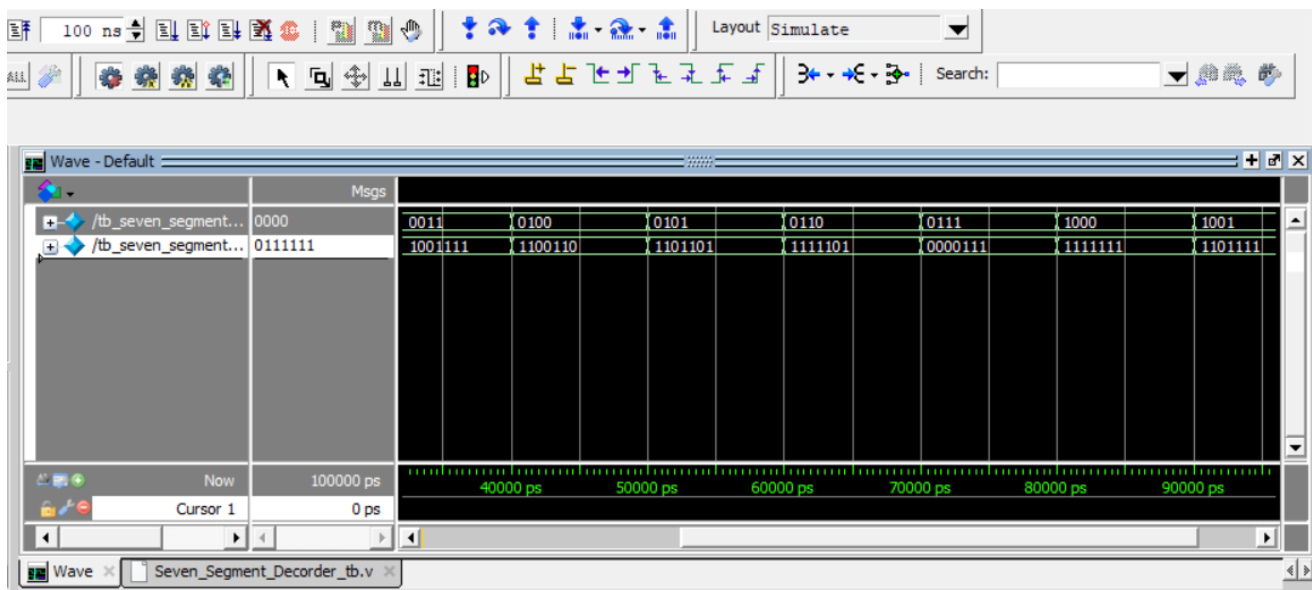
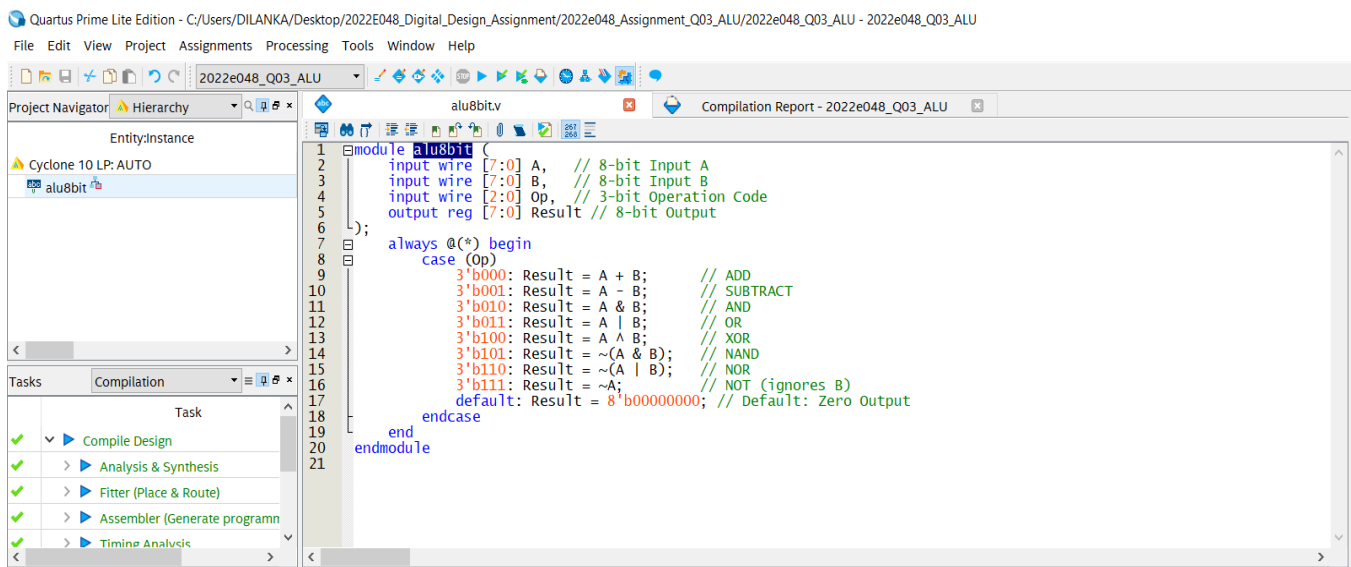


FIGURE 08: OUPUT FOR 7 SEGMENT DECODER

3. Design an 8-bit Arithmetic Logic Unit (ALU) using Verilog that performs the following operations based on a 3-bit operation code (Op [2:0]):

- ✦ Op = 000: ADD
- ✦ Op = 001: SUBTRACT
- ✦ Op = 010: AND
- ✦ Op = 011: OR
- ✦ Op = 100: XOR
- ✦ Op = 101: NAND
- ✦ Op = 110: NOR
- ✦ Op = 111: NOT The ALU should have two 8-bit inputs (A[7:0], B[7:0]) and an 8-bit output (Result[7:0]).



```
1 module alu8bit (
2     input wire [7:0] A, // 8-bit Input A
3     input wire [7:0] B, // 8-bit Input B
4     input wire [2:0] Op, // 3-bit Operation Code
5     output reg [7:0] Result // 8-bit Output
6 );
7
8 always @(*) begin
9     case (Op)
10        3'b000: Result = A + B; // ADD
11        3'b001: Result = A - B; // SUBTRACT
12        3'b010: Result = A & B; // AND
13        3'b011: Result = A | B; // OR
14        3'b100: Result = A ^ B; // XOR
15        3'b101: Result = ~(A & B); // NAND
16        3'b110: Result = ~(A | B); // NOR
17        3'b111: Result = ~A; // NOT (ignores B)
18        default: Result = 8'b00000000; // Default: Zero Output
19    endcase
20 end
21 endmodule
```

FIGURE 09: VERILOG CODE FOR 8 BIT ALU

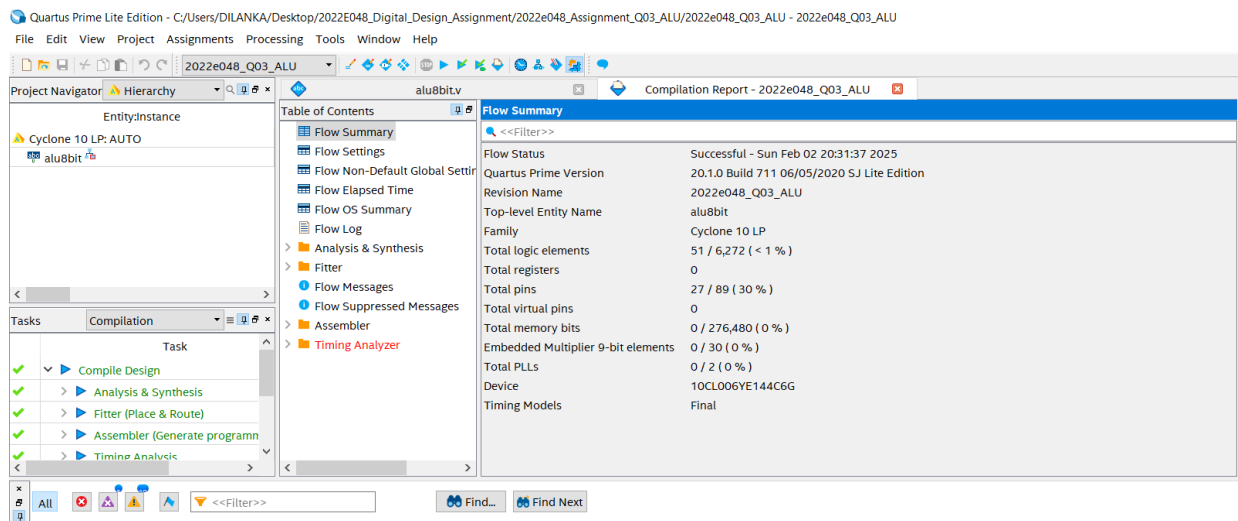


FIGURE 10: ANALYSIS REPORT FOR ALU-8 BIT

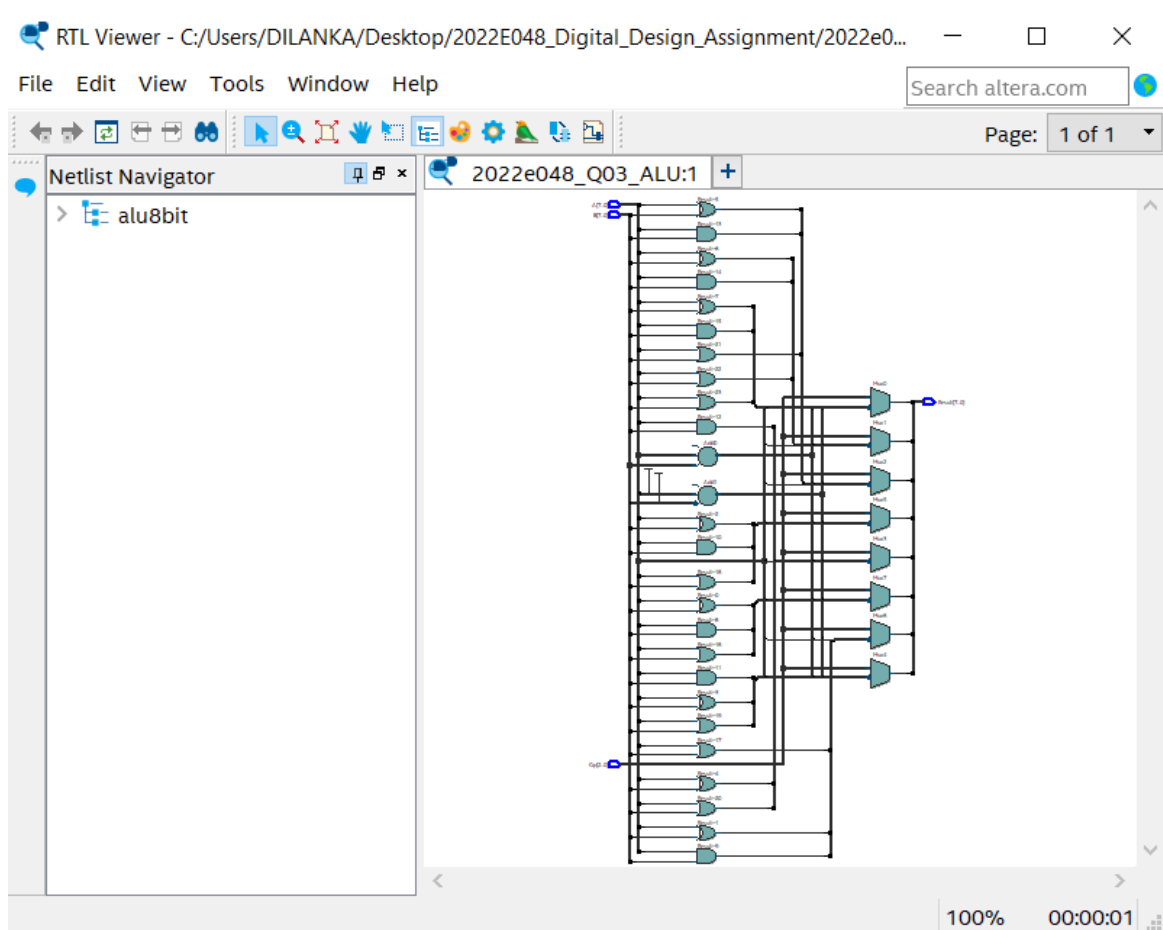


FIGURE 11: LOGIC CIRCUIT FOR 8 BIT ALU

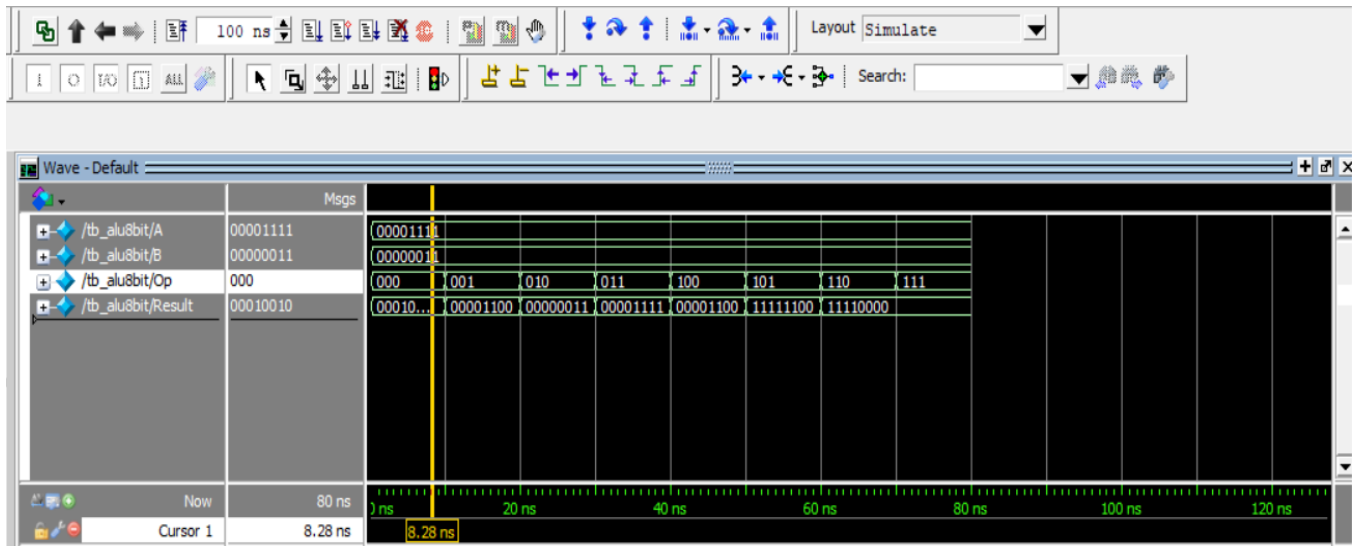


FIGURE 12: OUTPUT FOR 8 BIT ALU